

**TUGAS
CYBER PHYSICAL SYSTEM**

**IMPLEMENTASI KOMUNIKASI MQTT MENGGUNAKAN
PYTHON DAN MOSQUITTO BROKER PADA SISTEM SMART
ROOM MONITORING**



Oleh :
Nabilla Suci Amanda
Nim.235150301111044

FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA
MALANG
2026

DAFTAR ISI

DAFTAR ISI.....	1
A. PENDAHULUAN.....	2
B. DESAIN SISTEM.....	3
C. MODEL KOMUNIKASI MQTT.....	4
D. IMPLEMENTASI PROGRAM.....	5
1. Persiapan Lingkungan Sistem.....	5
2. Implementasi Skenario 1 : Komunikasi Dasar Publisher Subscriber.....	5
3. Implementasi Skenario 2 : Pengujian Quality of Service (QoS).....	7
4. Implementasi Skenario 3 : Penggunaan Beberapa Topik.....	8
5. Implementasi Skenario 4 : Wildcard Topic (+).....	10
6. Implementasi Skenario 5 : Wildcard Topic (#).....	12
E. PENGUJIAN DAN HASIL.....	15
1. Pengujian Skenario 1 : Komunikasi Dasar Publisher Subscriber.....	15
2. Pengujian Skenario 2 : Quality of Service (QoS).....	16
3. Pengujian Skenario 3 : Penggunaan Beberapa Topik.....	18
4. Pengujian Skenario 4 : Wildcard Topic (+).....	20
5. Pengujian Skenario 5 : Wildcard Topic (#).....	22
F. ANALISIS.....	25
G. KESIMPULAN.....	26

A. PENDAHULUAN

Cyber Physical System (CPS) merupakan sistem yang mengintegrasikan komponen komputasi (cyber) dengan proses fisik melalui mekanisme pemantauan, komunikasi, dan pengendalian secara berkelanjutan. Dalam implementasinya, CPS memanfaatkan sensor untuk memperoleh data dari lingkungan fisik, kemudian data tersebut diproses oleh sistem komputasi dan dapat digunakan untuk menghasilkan keputusan maupun tindakan tertentu. Integrasi antara komponen fisik dan digital ini banyak diterapkan pada berbagai bidang seperti smart home, smart agriculture, sistem industri, dan monitoring lingkungan.

Salah satu teknologi komunikasi yang banyak digunakan dalam sistem CPS dan Internet of Things (IoT) adalah Message Queuing Telemetry Transport (MQTT). MQTT merupakan protokol komunikasi ringan berbasis model publish subscribe yang dirancang untuk pertukaran data secara efisien melalui perantara berupa broker. Pada model ini, publisher bertugas mengirimkan data ke broker menggunakan topik tertentu, sedangkan subscriber menerima data berdasarkan topik yang telah di-subscribe tanpa harus terhubung secara langsung dengan publisher. Pola komunikasi ini memungkinkan proses pertukaran data menjadi lebih fleksibel, efisien, serta sesuai digunakan pada sistem monitoring yang melibatkan banyak perangkat.

Pada praktikum ini dilakukan implementasi komunikasi MQTT menggunakan bahasa pemrograman Python dan Mosquitto Broker dengan studi kasus Smart Room Monitoring. Sistem menggunakan data sensor virtual berupa suhu, kelembapan, dan intensitas cahaya yang dikirim melalui beberapa topik MQTT. Selain komunikasi dasar antara publisher dan subscriber, pengujian juga dilakukan menggunakan variasi Quality of Service (QoS) serta wildcard topic (+ dan #) untuk memahami bagaimana MQTT mengelola distribusi pesan pada berbagai kondisi komunikasi.

Tujuan dari praktikum ini adalah memahami konsep komunikasi MQTT dalam Cyber Physical System, mengimplementasikan publisher dan subscriber menggunakan Python, menjalankan Mosquitto Broker sebagai message broker, serta menganalisis pengaruh QoS dan struktur topik terhadap proses pengiriman dan penerimaan pesan.

B. DESAIN SISTEM

Sistem yang dirancang pada praktikum ini menggunakan konsep Smart Room Monitoring berbasis komunikasi MQTT dengan memanfaatkan Mosquitto sebagai broker serta Python sebagai publisher dan subscriber. Sistem menerapkan model komunikasi publish subscribe, di mana publisher bertugas mengirimkan data sensor virtual menuju broker, kemudian broker mendistribusikan pesan tersebut kepada subscriber yang telah melakukan subscribe pada topik tertentu. Pada implementasi ini, publisher berperan sebagai sensor virtual yang menghasilkan data kondisi ruangan berupa suhu (temperature), kelembapan (humidity), dan intensitas cahaya (light). Data tersebut dikirim ke Mosquitto Broker melalui beberapa topik MQTT yang telah ditentukan. Broker berfungsi sebagai perantara komunikasi yang menerima pesan dari publisher dan meneruskannya kepada subscriber sesuai dengan topik yang diakses.

Subscriber berperan sebagai aplikasi monitoring yang menerima serta menampilkan data yang diperoleh dari broker. Selain menerima pesan berdasarkan topik tertentu, subscriber juga diuji menggunakan wildcard topic (+ dan #) untuk mengamati bagaimana MQTT menangani distribusi pesan pada struktur topik yang berbeda. Struktur topik yang digunakan pada sistem Smart Room Monitoring ditunjukkan sebagai berikut:

- smartroom/room1/temperature
- smartroom/room1/humidity
- smartroom/room1/light

Selain topik dasar tersebut, pengujian wildcard dilakukan menggunakan:

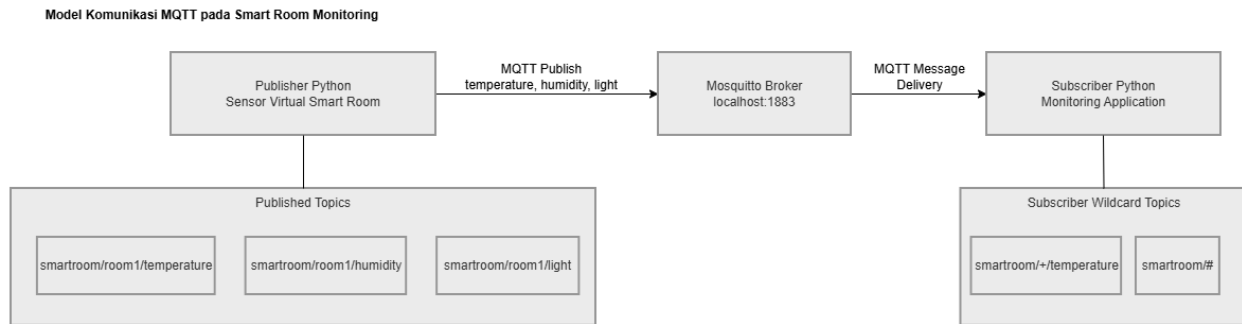
- smartroom/+ /temperature
- smartroom/#

Wildcard + digunakan untuk merepresentasikan satu level topik, sehingga subscriber dapat menerima seluruh data temperature dari berbagai ruangan tanpa perlu melakukan subscribe satu per satu. Sementara itu, wildcard # digunakan untuk menerima seluruh subtopik di bawah smartroom, sehingga subscriber dapat memantau seluruh data sistem secara menyeluruh. Desain sistem komunikasi yang diterapkan terdiri dari tiga komponen utama, yaitu:

- **Publisher Python (Sensor Virtual Smart Room)**
Publisher menghasilkan data virtual berupa temperature, humidity, dan light, kemudian mengirimkan data tersebut ke broker melalui topik MQTT tertentu.
- **Mosquitto Broker (localhost:1883)**
Broker bertugas menerima pesan dari publisher dan mendistribusikannya kepada subscriber sesuai topik yang telah di-subscribe.
- **Subscriber Python (Monitoring Application)**

Subscriber menerima pesan dari broker dan menampilkan data monitoring berdasarkan topik maupun wildcard yang digunakan.

C. MODEL KOMUNIKASI MQTT



Gambar 1. Model Komunikasi MQTT pada Smart Room Monitoring

Gambar 1 menunjukkan model komunikasi MQTT pada sistem Smart Room Monitoring yang dibangun menggunakan Python dan Mosquitto Broker. Publisher berperan sebagai sensor virtual yang menghasilkan data temperature, humidity, dan light kemudian mengirimkannya ke broker melalui mekanisme MQTT publish. Mosquitto Broker bertugas menerima dan mendistribusikan pesan kepada subscriber sesuai topik yang di-subscribe.

Subscriber berfungsi sebagai aplikasi monitoring yang menerima pesan berdasarkan topik tertentu maupun wildcard topic. Pada sistem ini digunakan wildcard + untuk menerima data temperature dari beberapa ruangan dan wildcard # untuk menerima seluruh data pada domain smartroom. Model komunikasi ini menunjukkan pola publish subscribe pada MQTT yang memungkinkan pertukaran data berlangsung secara fleksibel tanpa hubungan langsung antara publisher dan subscriber.

D. IMPLEMENTASI PROGRAM

Implementasi sistem dilakukan menggunakan bahasa pemrograman Python dengan bantuan library paho-mqtt serta Mosquitto sebagai MQTT broker. Program yang dikembangkan terdiri dari beberapa publisher dan subscriber yang digunakan untuk memenuhi lima skenario pengujian, yaitu komunikasi dasar publisher subscriber, pengujian Quality of Service (QoS), penggunaan beberapa topik, wildcard +, dan wildcard #.

1. Persiapan Lingkungan Sistem

Sebelum program dijalankan, dilakukan instalasi Python, Mosquitto Broker, serta library paho-mqtt. Mosquitto digunakan sebagai message broker yang menerima dan mendistribusikan pesan MQTT, sedangkan library paho-mqtt memungkinkan program Python melakukan komunikasi menggunakan protokol MQTT. Instalasi library dilakukan menggunakan perintah berikut:

```
pip install paho-mqtt
```

Broker Mosquitto dijalankan pada localhost dengan port default MQTT yaitu 1883.

2. Implementasi Skenario 1 : Komunikasi Dasar Publisher Subscriber

Pada skenario pertama dibuat komunikasi dasar antara publisher dan subscriber menggunakan topik smartroom/room1/temperature, Publisher mengirimkan data suhu virtual menuju broker, sedangkan subscriber menerima dan menampilkan pesan yang dikirimkan. Skenario ini bertujuan untuk membuktikan bahwa komunikasi dasar publish subscribe dapat berjalan dengan baik melalui Mosquitto Broker.

a. Subscriber Basic

```
import paho.mqtt.client as mqtt

# Fungsi saat berhasil connect ke broker
def on_connect(client, userdata, flags, reason_code,
properties=None):
    print("Connected to broker")
    client.subscribe("smartroom/room1/temperature")

# Fungsi saat menerima pesan
def on_message(client, userdata, msg):
    print(f"Topic: {msg.topic}")
    print(f"Message: {msg.payload.decode()}")
```

```
# Membuat client MQTT
client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)

# Menghubungkan callback
client.on_connect = on_connect
client.on_message = on_message

# Connect ke broker lokal
client.connect("localhost", 1883, 60)

# Menunggu pesan terus menerus
client.loop_forever()
```

b. Publisher Basic

```
import paho.mqtt.client as mqtt
import time

# Membuat client MQTT
client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)

# Connect ke broker
client.connect("localhost", 1883, 60)

# Data yang akan dikirim
temperature = "27 C"

# Publish pesan
client.publish(
    "smartroom/room1/temperature",
    temperature
)

print("Message sent:", temperature)

# Delay kecil agar publish selesai
time.sleep(1)

# Disconnect
client.disconnect()
```

3. Implementasi Skenario 2 : Pengujian Quality of Service (QoS)

Pada skenario kedua dilakukan pengujian menggunakan tiga level Quality of Service (QoS), yaitu QoS 0, QoS 1, dan QoS 2.

- QoS 0 (At Most Once)
Pesan dikirim satu kali tanpa jaminan penerimaan.
- QoS 1 (At Least Once)
Pesan dijamin sampai ke penerima, namun memungkinkan terjadinya duplikasi pesan.
- QoS 2 (Exactly Once)
Pesan dijamin diterima tepat satu kali melalui mekanisme handshake yang lebih kompleks.

Pengujian dilakukan pada topik smartroom/room1/qos, Publisher mengirimkan tiga pesan dengan level QoS berbeda dan subscriber menampilkan QoS dari pesan yang diterima.

a. Subscriber QoS

```
import paho.mqtt.client as mqtt

def on_connect(client, userdata, flags, reason_code,
properties=None):
    print("Connected to broker")
    client.subscribe("smartroom/room1/qos", qos=2)
    print("Subscribed to topic: smartroom/room1/qos")

def on_message(client, userdata, msg):
    print("-----")
    print(f"Topic    : {msg.topic}")
    print(f"QoS      : {msg.qos}")
    print(f"Message   : {msg.payload.decode()}")

client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)

client.on_connect = on_connect
client.on_message = on_message

client.connect("localhost", 1883, 60)
client.loop_forever()
```


b. Publisher QoS

```
import paho.mqtt.client as mqtt

client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
client.connect("localhost", 1883, 60)
client.loop_start()

msg0 = client.publish("smartroom/room1/qos", "Temperature 27 C -
QoS 0", qos=0)
msg0.wait_for_publish()
print("Sent QoS 0")

msg1 = client.publish("smartroom/room1/qos", "Temperature 27 C -
QoS 1", qos=1)
msg1.wait_for_publish()
print("Sent QoS 1")

msg2 = client.publish("smartroom/room1/qos", "Temperature 27 C -
QoS 2", qos=2)
msg2.wait_for_publish()
print("Sent QoS 2")

client.loop_stop()
client.disconnect()
```

4. Implementasi Skenario 3 : Penggunaan Beberapa Topik

Pada skenario ini publisher mengirimkan beberapa data sensor virtual secara terpisah menggunakan beberapa topik MQTT, yaitu smartroom/room1/temperature, smartroom/room1/humidity, smartroom/room1/light, Subscriber melakukan subscribe terhadap seluruh topik tersebut dan menampilkan data yang diterima. Pengujian ini menunjukkan bahwa MQTT mampu menangani beberapa jenis data dalam satu sistem monitoring.

a. Subscriber Multi Topic

```
import paho.mqtt.client as mqtt
```

```
def on_connect(client, userdata, flags, reason_code,
properties=None):
    print("Connected to broker")

    client.subscribe("smartroom/room1/temperature")
    client.subscribe("smartroom/room1/humidity")
    client.subscribe("smartroom/room1/light")

    print("Subscribed to multiple topics")

def on_message(client, userdata, msg):
    print("-----")
    print(f"Topic    : {msg.topic}")
    print(f"Message  : {msg.payload.decode()}")

client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)

client.on_connect = on_connect
client.on_message = on_message

client.connect("localhost", 1883, 60)

client.loop_forever()
```

b. Publisher Multi Topic

```
import paho.mqtt.client as mqtt
import time

client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)

client.connect("localhost", 1883, 60)
client.loop_start()

# Temperature
client.publish(
    "smartroom/room1/temperature",
    "28 C"
)

print("Temperature sent")
```

```
time.sleep(1)

# Humidity
client.publish(
    "smartroom/room1/humidity",
    "65 %"
)
print("Humidity sent")
time.sleep(1)

# Light
client.publish(
    "smartroom/room1/light",
    "Bright"
)
print("Light sent")

time.sleep(1)

client.loop_stop()
client.disconnect()
```

5. Implementasi Skenario 4 : Wildcard Topic (+)

Pada skenario keempat digunakan wildcard + dengan topik smartroom/+/temperature, Wildcard + digunakan untuk mewakili satu level topik sehingga subscriber dapat menerima seluruh data temperature dari beberapa ruangan tanpa perlu melakukan subscribe pada setiap topik secara terpisah. Publisher mengirimkan data temperature dan humidity, sedangkan subscriber hanya menerima data temperature yang sesuai dengan pola wildcard.

a. Subscriber Wildcard Plus

```
import paho.mqtt.client as mqtt

def on_connect(client, userdata, flags, reason_code,
properties=None):
    print("Connected to broker")
    client.subscribe("smartroom/+/temperature")
    print("Subscribed to topic: smartroom/+/temperature")
```

```
def on_message(client, userdata, msg):  
    print("-----")  
    print(f"Topic    : {msg.topic}")  
    print(f"Message  : {msg.payload.decode()}")  
  
client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)  
  
client.on_connect = on_connect  
client.on_message = on_message  
  
client.connect("localhost", 1883, 60)  
  
client.loop_forever()
```

b. Publisher Wildcard Plus

```
import paho.mqtt.client as mqtt  
import time  
  
client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)  
  
client.connect("localhost", 1883, 60)  
client.loop_start()  
  
client.publish("smartroom/room1/temperature", "Room 1 Temperature:  
28 C")  
print("Room 1 temperature sent")  
time.sleep(1)  
  
client.publish("smartroom/room2/temperature", "Room 2 Temperature:  
30 C")  
print("Room 2 temperature sent")  
time.sleep(1)  
  
client.publish("smartroom/room1/humidity", "Room 1 Humidity: 65  
%")  
print("Room 1 humidity sent")  
  
time.sleep(1)
```

```
client.loop_stop()
client.disconnect()
```

6. Implementasi Skenario 5 : Wildcard Topic (#)

Pada skenario terakhir digunakan wildcard # dengan topik smartroom/#. Wildcard # memungkinkan subscriber menerima seluruh subtopik di bawah domain smartroom. Dengan mekanisme ini, subscriber dapat memonitor seluruh data yang dikirim oleh publisher tanpa perlu menentukan topik satu per satu. Publisher mengirimkan beberapa data berupa temperature, humidity, dan light dari beberapa topik, kemudian subscriber menampilkan seluruh pesan yang diterima.

a. Subscriber Wildcard Hash

```
import paho.mqtt.client as mqtt

def on_connect(client, userdata, flags, reason_code,
properties=None):
    print("Connected to broker")
    client.subscribe("smartroom/#")
    print("Subscribed to topic: smartroom/#")

def on_message(client, userdata, msg):
    print("-----")
    print(f"Topic    : {msg.topic}")
    print(f"Message  : {msg.payload.decode()}")

client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)

client.on_connect = on_connect
client.on_message = on_message

client.connect("localhost", 1883, 60)

client.loop_forever()
```

b. Publisher Wildcard Hash

```
import paho.mqtt.client as mqtt
```

```
import time

client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)

client.connect("localhost", 1883, 60)
client.loop_start()

client.publish(
    "smartroom/room1/temperature",
    "Room 1 Temperature: 28 C"
)
print("Temperature sent")
time.sleep(1)

client.publish(
    "smartroom/room1/humidity",
    "Room 1 Humidity: 65 %"
)
print("Humidity sent")
time.sleep(1)

client.publish(
    "smartroom/room1/light",
    "Room 1 Light: Bright"
)
print("Light sent")
time.sleep(1)

client.publish(
    "smartroom/room2/temperature",
    "Room 2 Temperature: 30 C"
)
print("Room 2 temperature sent")

time.sleep(1)

client.loop_stop()
client.disconnect()
```

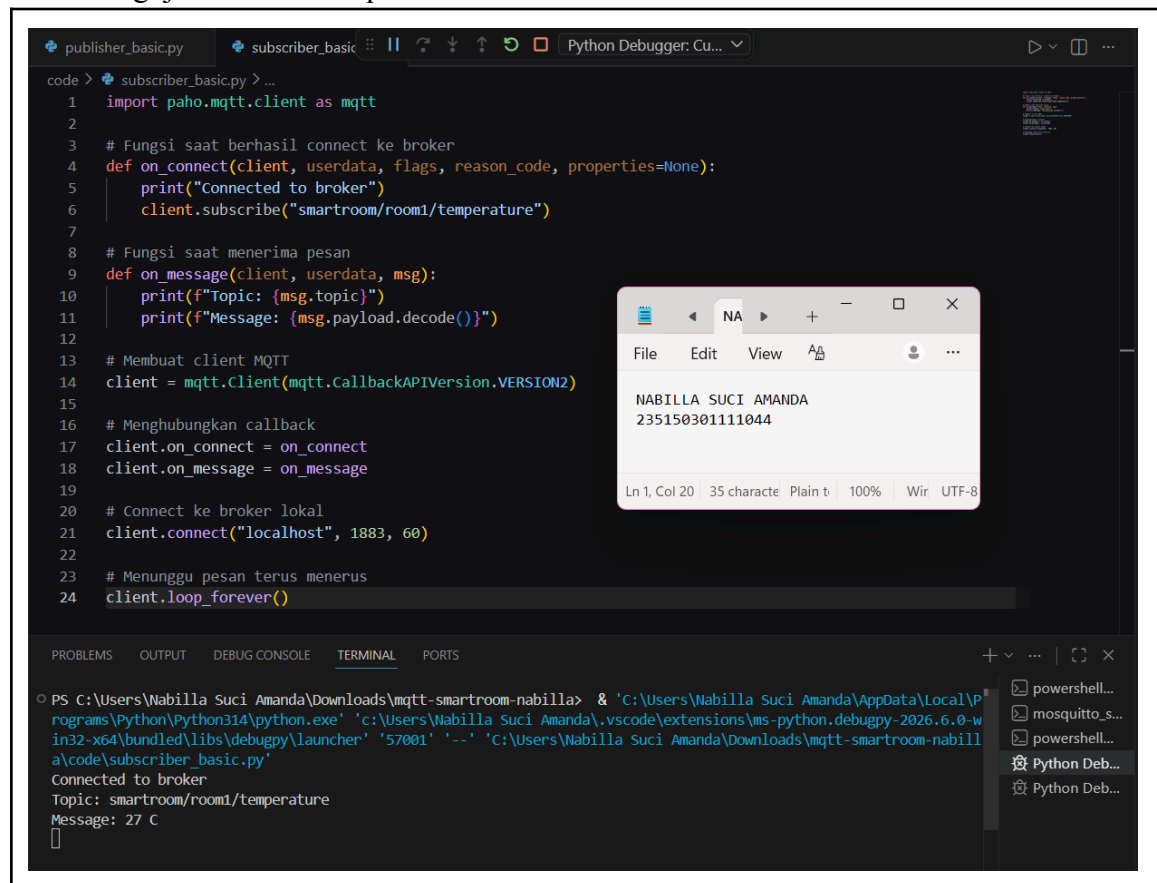
E. PENGUJIAN DAN HASIL

Pengujian dilakukan untuk memastikan komunikasi MQTT menggunakan Python dan Mosquitto Broker dapat berjalan sesuai dengan skenario yang telah ditentukan. Setiap skenario diuji menggunakan kombinasi publisher dan subscriber yang berbeda untuk mengamati proses pengiriman, distribusi, serta penerimaan pesan melalui broker MQTT.

1. Pengujian Skenario 1 : Komunikasi Dasar Publisher Subscriber

Pada pengujian pertama dilakukan komunikasi dasar menggunakan topik smartroom/room1/temperature. Publisher mengirimkan data suhu virtual sebesar 27 C menuju broker, kemudian subscriber menerima dan menampilkan pesan yang dikirimkan. Hasil pengujian menunjukkan bahwa subscriber berhasil menerima pesan dari publisher melalui Mosquitto Broker. Hal ini membuktikan bahwa mekanisme dasar publish subscribe menggunakan MQTT telah berjalan dengan baik.

a. Hasil Pengujian Subscriber pada Skenario 1 : Komunikasi Dasar Publisher-Subscriber



The screenshot displays a Python IDE with a file named `subscriber_basic.py`. The code defines an MQTT client that connects to a local broker at `localhost:1883` and subscribes to the topic `smartroom/room1/temperature`. Upon receiving a message, it prints the topic and the decoded message. The terminal output shows the client successfully connecting and receiving the message `27 C`. A secondary window shows the received message details: `NABILLA SUCI AMANDA` and `235150301111044`.

```

code > subscriber_basic.py > ...
1  import paho.mqtt.client as mqtt
2
3  # Fungsi saat berhasil connect ke broker
4  def on_connect(client, userdata, flags, reason_code, properties=None):
5      print("Connected to broker")
6      client.subscribe("smartroom/room1/temperature")
7
8  # Fungsi saat menerima pesan
9  def on_message(client, userdata, msg):
10     print(f"Topic: {msg.topic}")
11     print(f"Message: {msg.payload.decode()}")
12
13 # Membuat client MQTT
14 client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
15
16 # Menghubungkan callback
17 client.on_connect = on_connect
18 client.on_message = on_message
19
20 # Connect ke broker lokal
21 client.connect("localhost", 1883, 60)
22
23 # Menunggu pesan terus menerus
24 client.loop_forever()

```

Terminal Output:

```

PS C:\Users\Nabilla Suci Amanda\Downloads\mqtt-smartroom-nabilla> & 'C:\Users\Nabilla Suci Amanda\AppData\Local\Programs\Python\Python314\python.exe' 'C:\Users\Nabilla Suci Amanda\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '57001' '--' 'C:\Users\Nabilla Suci Amanda\Downloads\mqtt-smartroom-nabilla\code\subscriber_basic.py'
Connected to broker
Topic: smartroom/room1/temperature
Message: 27 C

```

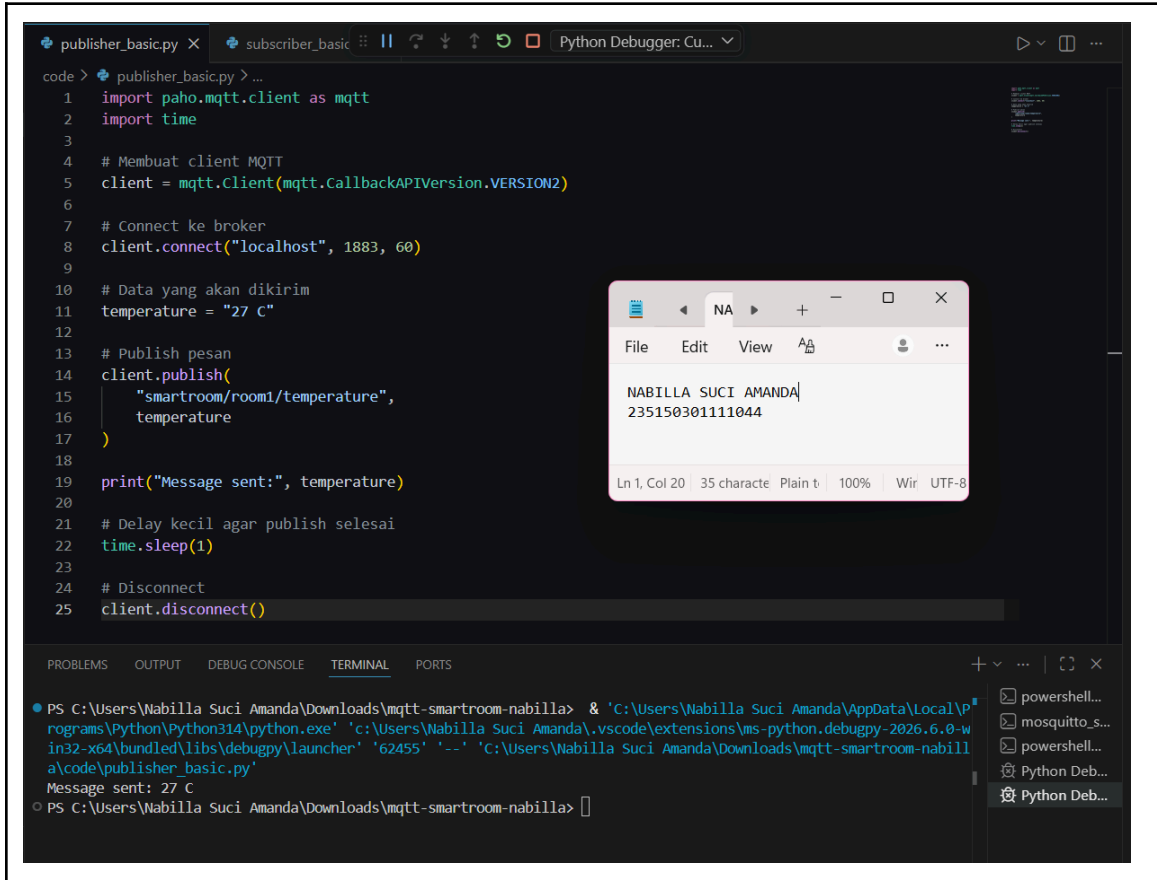
Message Details Window:

```

NABILLA SUCI AMANDA
235150301111044

```

b. Hasil Pengujian Publisher pada Skenario 1 : Komunikasi Dasar Publisher Subscriber



```
code > publisher_basic.py > ...
1  import paho.mqtt.client as mqtt
2  import time
3
4  # Membuat client MQTT
5  client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
6
7  # Connect ke broker
8  client.connect("localhost", 1883, 60)
9
10 # Data yang akan dikirim
11 temperature = "27 C"
12
13 # Publish pesan
14 client.publish(
15     "smartroom/room1/temperature",
16     temperature
17 )
18
19 print("Message sent:", temperature)
20
21 # Delay kecil agar publish selesai
22 time.sleep(1)
23
24 # Disconnect
25 client.disconnect()
```

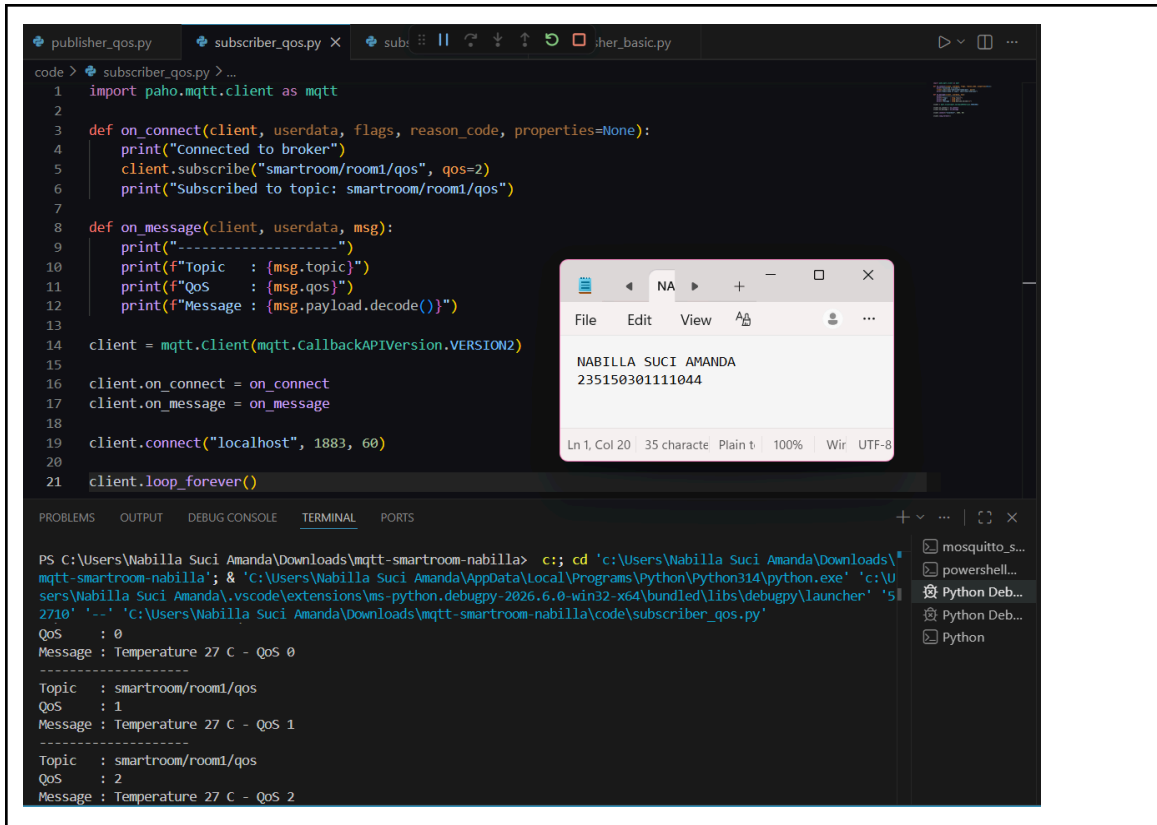
Terminal Output:

```
PS C:\Users\Nabilla Suci Amanda\Downloads\mqtt-smartroom-nabilla> & 'C:\Users\Nabilla Suci Amanda\AppData\Local\Programs\Python\Python314\python.exe' 'C:\Users\Nabilla Suci Amanda\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '62455' '--' 'C:\Users\Nabilla Suci Amanda\Downloads\mqtt-smartroom-nabilla\code\publisher_basic.py'
Message sent: 27 C
PS C:\Users\Nabilla Suci Amanda\Downloads\mqtt-smartroom-nabilla>
```

2. Pengujian Skenario 2 : Quality of Service (QoS)

Pada pengujian kedua dilakukan pengiriman pesan menggunakan tiga level Quality of Service (QoS), yaitu QoS 0, QoS 1, dan QoS 2. Topik yang digunakan adalah smartroom/room1/qos. Publisher mengirimkan tiga pesan dengan level QoS berbeda, kemudian subscriber menampilkan QoS dari setiap pesan yang diterima. Hasil pengujian menunjukkan bahwa seluruh pesan berhasil diterima oleh subscriber dengan level QoS yang sesuai. QoS 0 berhasil mengirim pesan tanpa mekanisme konfirmasi, QoS 1 menjamin pesan sampai minimal satu kali, sedangkan QoS 2 berhasil diterima tepat satu kali melalui mekanisme handshake yang lebih lengkap.

a. Hasil Pengujian Subscriber pada Skenario 2 : Quality of Service (QoS)



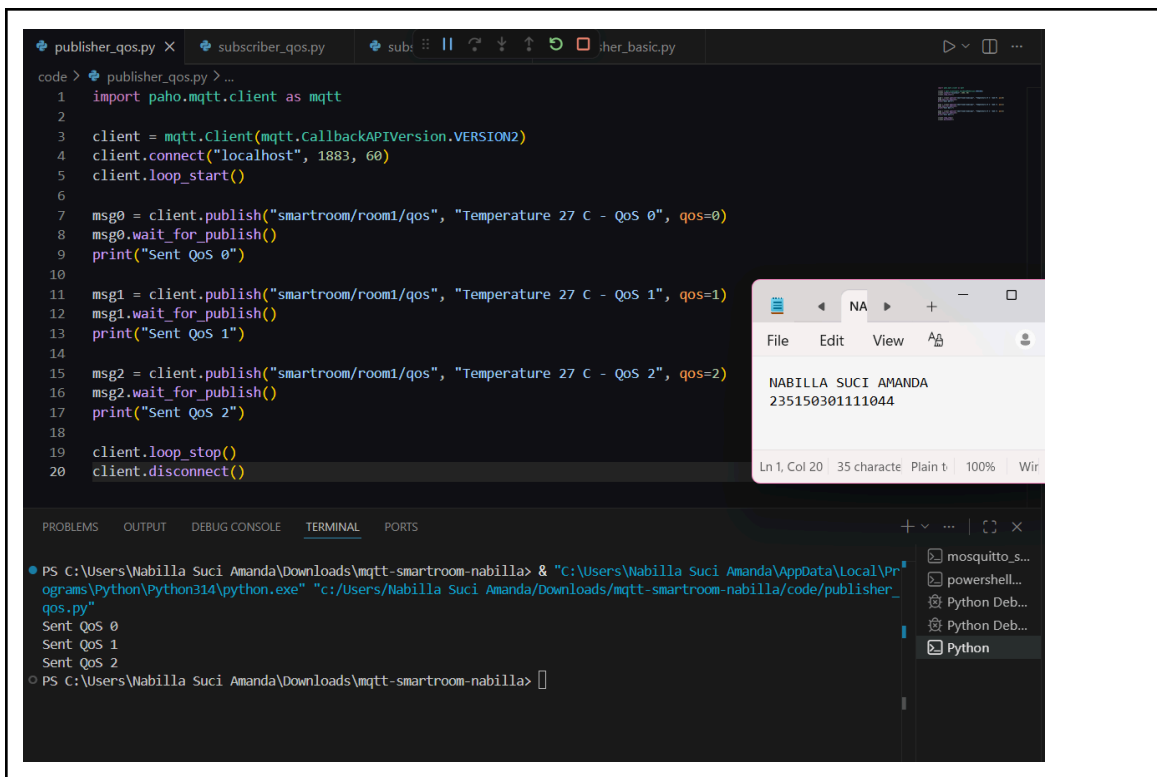
```

1 import paho.mqtt.client as mqtt
2
3 def on_connect(client, userdata, flags, reason_code, properties=None):
4     print("Connected to broker")
5     client.subscribe("smartroom/room1/qos", qos=2)
6     print("Subscribed to topic: smartroom/room1/qos")
7
8 def on_message(client, userdata, msg):
9     print("-----")
10    print(f"Topic : {msg.topic}")
11    print(f"QoS : {msg.qos}")
12    print(f"Message : {msg.payload.decode()}")
13
14 client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
15
16 client.on_connect = on_connect
17 client.on_message = on_message
18
19 client.connect("localhost", 1883, 60)
20
21 client.loop_forever()
    
```

```

PS C:\Users\Nabilla Suci Amanda\Downloads\mqtt-smartroom-nabilla> c:: cd 'c:\Users\Nabilla Suci Amanda\Downloads\mqtt-smartroom-nabilla'; & "c:\Users\Nabilla Suci Amanda\AppData\Local\Programs\Python\Python314\python.exe" 'c:\Users\Nabilla Suci Amanda\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '512710' '-.' 'c:\Users\Nabilla Suci Amanda\Downloads\mqtt-smartroom-nabilla\code\subscriber_qos.py'
QoS : 0
Message : Temperature 27 C - QoS 0
-----
Topic : smartroom/room1/qos
QoS : 1
Message : Temperature 27 C - QoS 1
-----
Topic : smartroom/room1/qos
QoS : 2
Message : Temperature 27 C - QoS 2
    
```

b. Hasil Pengujian Publisher pada Skenario 2 : Quality of Service (QoS)



```

1 import paho.mqtt.client as mqtt
2
3 client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
4 client.connect("localhost", 1883, 60)
5 client.loop_start()
6
7 msg0 = client.publish("smartroom/room1/qos", "Temperature 27 C - QoS 0", qos=0)
8 msg0.wait_for_publish()
9 print("Sent QoS 0")
10
11 msg1 = client.publish("smartroom/room1/qos", "Temperature 27 C - QoS 1", qos=1)
12 msg1.wait_for_publish()
13 print("Sent QoS 1")
14
15 msg2 = client.publish("smartroom/room1/qos", "Temperature 27 C - QoS 2", qos=2)
16 msg2.wait_for_publish()
17 print("Sent QoS 2")
18
19 client.loop_stop()
20 client.disconnect()
    
```

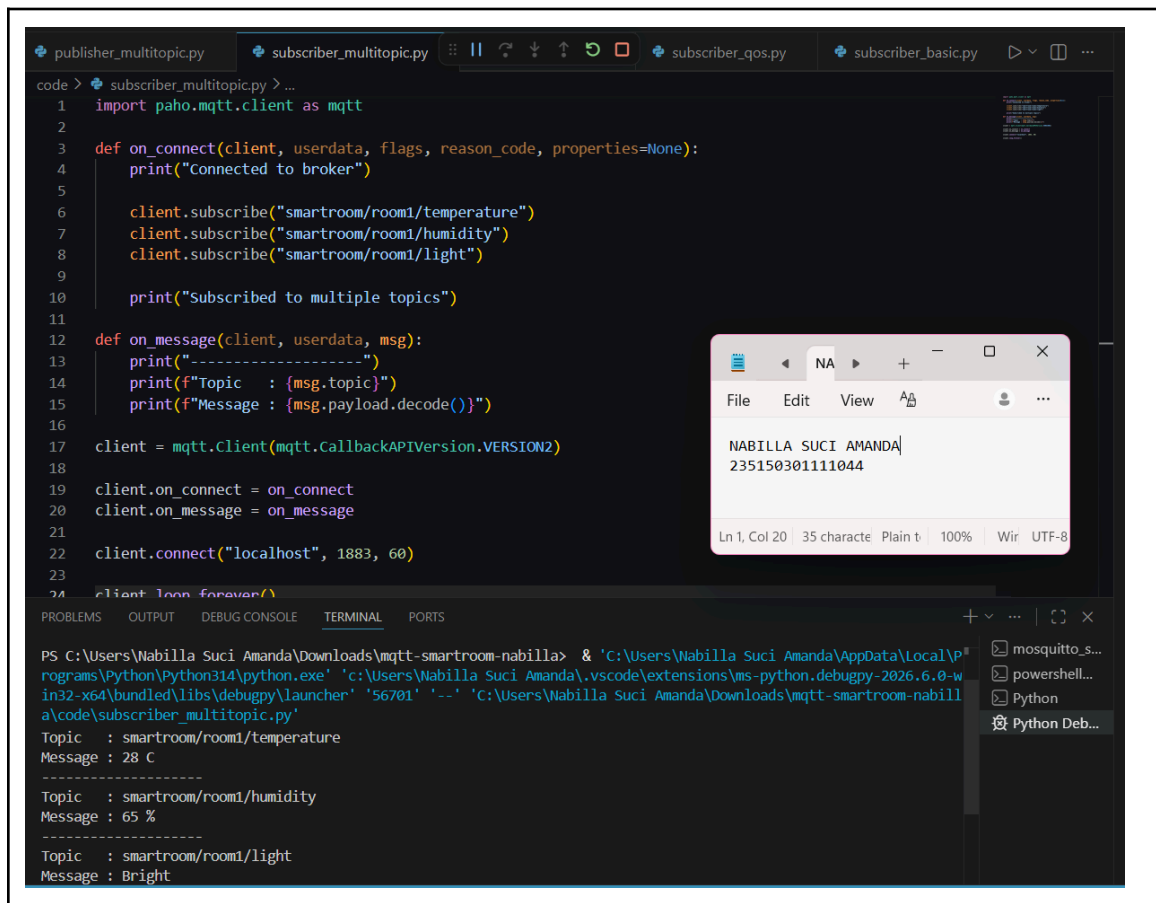
```

PS C:\Users\Nabilla Suci Amanda\Downloads\mqtt-smartroom-nabilla> "c:\Users\Nabilla Suci Amanda\AppData\Local\Programs\Python\Python314\python.exe" "c:\Users\Nabilla Suci Amanda\Downloads\mqtt-smartroom-nabilla\code\publisher_qos.py"
Sent QoS 0
Sent QoS 1
Sent QoS 2
PS C:\Users\Nabilla Suci Amanda\Downloads\mqtt-smartroom-nabilla>
    
```

3. Pengujian Skenario 3 : Penggunaan Beberapa Topik

Pada pengujian ini publisher mengirimkan beberapa data sensor virtual melalui topik yang berbeda, yaitu smartroom/room1/temperature, smartroom/room1/humidity, smartroom/room1/light. Subscriber melakukan subscribe pada seluruh topik tersebut dan menampilkan data yang diterima. Hasil pengujian menunjukkan bahwa subscriber berhasil menerima seluruh data yang dikirimkan publisher, yaitu temperature, humidity, dan light. Pengujian ini membuktikan bahwa MQTT mampu menangani komunikasi dengan beberapa topik dalam satu sistem monitoring.

a. Hasil Pengujian Subscriber pada Skenario 3 : Penggunaan Beberapa Topik



The screenshot displays a Visual Studio Code editor with a Python script named `subscriber_multitopic.py` and its execution output in the terminal.

Script Code:

```

1 import paho.mqtt.client as mqtt
2
3 def on_connect(client, userdata, flags, reason_code, properties=None):
4     print("Connected to broker")
5
6     client.subscribe("smartroom/room1/temperature")
7     client.subscribe("smartroom/room1/humidity")
8     client.subscribe("smartroom/room1/light")
9
10    print("Subscribed to multiple topics")
11
12    def on_message(client, userdata, msg):
13        print("-----")
14        print(f"Topic : {msg.topic}")
15        print(f"Message : {msg.payload.decode()}")
16
17    client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
18
19    client.on_connect = on_connect
20    client.on_message = on_message
21
22    client.connect("localhost", 1883, 60)
23
24    client.loop_forever()

```

Terminal Output:

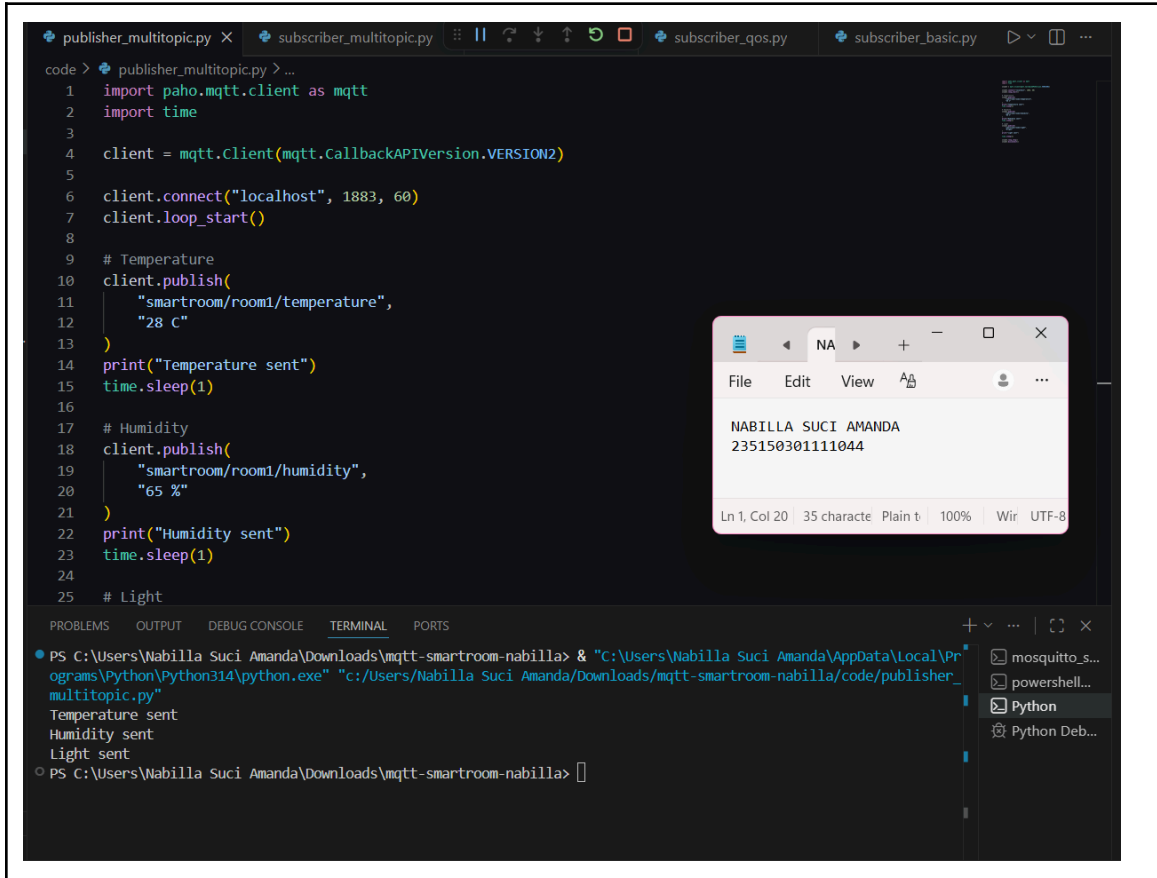
```

PS C:\Users\Nabilla Suci Amanda\Downloads\mqtt-smartroom-nabilla> & 'C:\Users\Nabilla Suci Amanda\AppData\Local\Programs\Python\Python314\python.exe' 'C:\Users\Nabilla Suci Amanda\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '56701' '--' 'C:\Users\Nabilla Suci Amanda\Downloads\mqtt-smartroom-nabilla\code\subscriber_multitopic.py'
Topic : smartroom/room1/temperature
Message : 28 C
-----
Topic : smartroom/room1/humidity
Message : 65 %
-----
Topic : smartroom/room1/light
Message : Bright

```

The terminal also shows a list of running processes on the right, including `mosquitto_s...`, `powershell...`, `Python`, and `Python Deb...`.

b. Hasil Pengujian Publisher pada Skenario 3 : Penggunaan Beberapa Topik



The screenshot shows a VS Code editor with a Python script named `publisher_multitopic.py`. The script uses the `paho-mqtt` library to connect to a local MQTT broker on `localhost:1883`. It publishes three topics: `smartroom/room1/temperature` (28 C), `smartroom/room1/humidity` (65 %), and `smartroom/room1/light`. A terminal window at the bottom shows the execution of the script, displaying the output: `Temperature sent`, `Humidity sent`, and `Light sent`. A small window titled "NA" is also visible, displaying the text: `NABILLA SUCI AMANDA` and `235150301111044`.

```

code > publisher_multitopic.py > ...
1  import paho.mqtt.client as mqtt
2  import time
3
4  client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
5
6  client.connect("localhost", 1883, 60)
7  client.loop_start()
8
9  # Temperature
10 client.publish(
11     "smartroom/room1/temperature",
12     "28 C"
13 )
14 print("Temperature sent")
15 time.sleep(1)
16
17 # Humidity
18 client.publish(
19     "smartroom/room1/humidity",
20     "65 %"
21 )
22 print("Humidity sent")
23 time.sleep(1)
24
25 # Light

```

PS C:\Users\Nabilla Suci Amanda\Downloads\mqtt-smartroom-nabilla> & "C:\Users\Nabilla Suci Amanda\AppData\Local\Programs\Python\Python314\python.exe" "C:\Users\Nabilla Suci Amanda\Downloads\mqtt-smartroom-nabilla\code\publisher_multitopic.py"

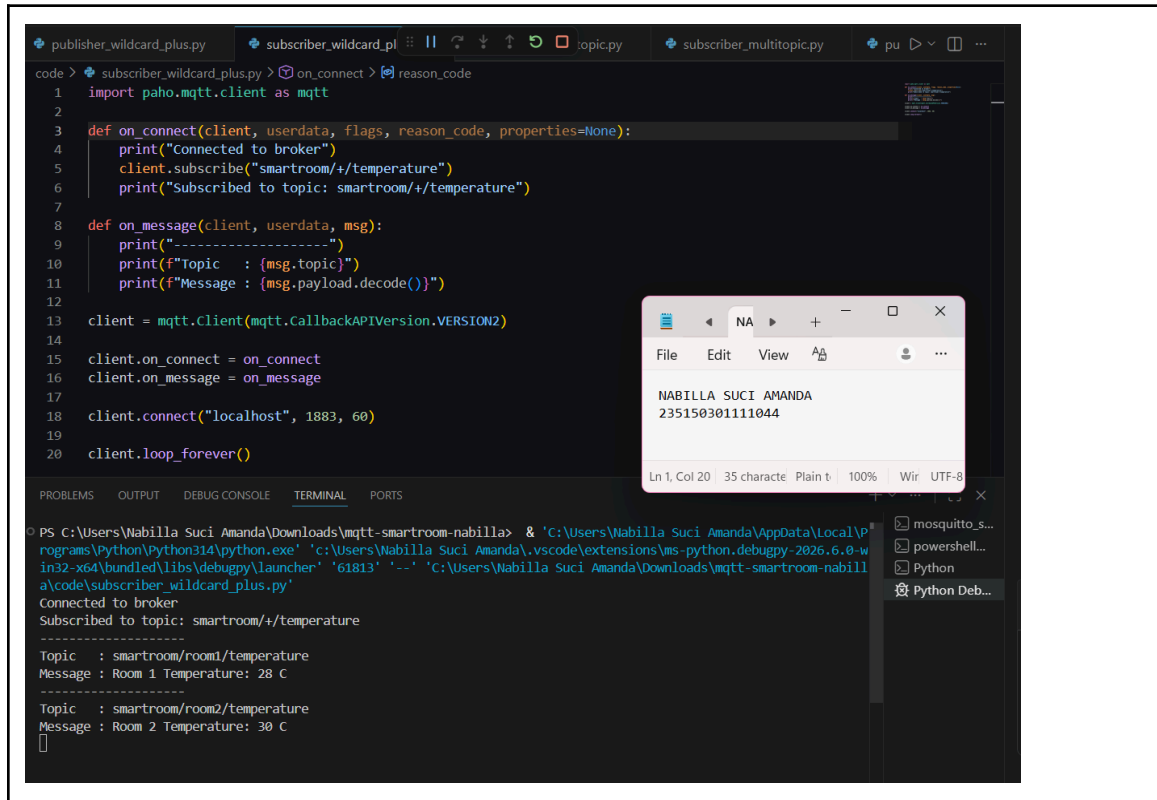
Temperature sent
Humidity sent
Light sent

PS C:\Users\Nabilla Suci Amanda\Downloads\mqtt-smartroom-nabilla>

4. Pengujian Skenario 4 : Wildcard Topic (+)

Pengujian keempat menggunakan wildcard `+` dengan topik subscribe `smartroom/+ /temperature`. Publisher mengirimkan data temperature dan humidity dari beberapa topik, sedangkan subscriber hanya menerima pesan yang sesuai dengan pola wildcard. Hasil pengujian menunjukkan bahwa subscriber berhasil menerima data `smartroom/room1/temperature`, `smartroom/room2/temperature`. Namun subscriber tidak menerima data humidity karena wildcard `+` hanya menggantikan satu level topik dan tetap mensyaratkan level terakhir berupa temperature.

a. Hasil Pengujian Subscriber pada Skenario 4 : Wildcard Topic (+)



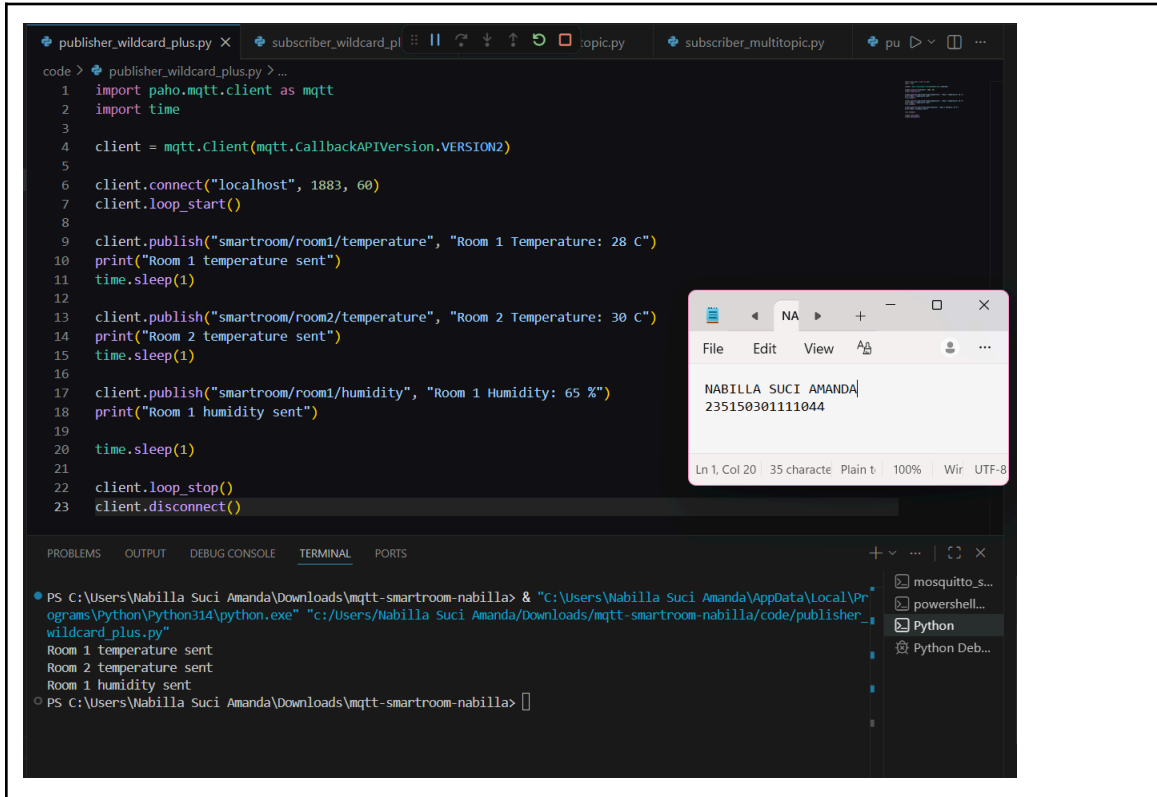
The screenshot displays a Visual Studio Code editor with a Python script named `subscriber_wildcard_plus.py` and its execution output in the terminal. The script uses the `paho-mqtt` library to connect to a local MQTT broker and subscribe to a wildcard topic `smartroom/+/temperature`. The terminal output shows the successful connection and the receipt of two messages from different topics.

```
code > subscriber_wildcard_plus.py on_connect > reason_code
1 import paho.mqtt.client as mqtt
2
3 def on_connect(client, userdata, flags, reason_code, properties=None):
4     print("Connected to broker")
5     client.subscribe("smartroom/+/temperature")
6     print("Subscribed to topic: smartroom/+/temperature")
7
8 def on_message(client, userdata, msg):
9     print("-----")
10    print(f"Topic : {msg.topic}")
11    print(f"Message : {msg.payload.decode()}")
12
13 client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
14
15 client.on_connect = on_connect
16 client.on_message = on_message
17
18 client.connect("localhost", 1883, 60)
19
20 client.loop_forever()
```

Terminal Output:

```
PS C:\Users\Nabilla Suci Amanda\Downloads\mqtt-smartroom-nabilla> & 'C:\Users\Nabilla Suci Amanda\AppData\Local\Programs\Python\Python314\python.exe' 'C:\Users\Nabilla Suci Amanda\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '61813' '--' 'C:\Users\Nabilla Suci Amanda\Downloads\mqtt-smartroom-nabilla\code\subscriber_wildcard_plus.py'
Connected to broker
Subscribed to topic: smartroom/+/temperature
-----
Topic : smartroom/room1/temperature
Message : Room 1 Temperature: 28 C
-----
Topic : smartroom/room2/temperature
Message : Room 2 Temperature: 30 C
[]
```

b. Hasil Pengujian Publisher pada Skenario 4 : Wildcard Topic (+)



The screenshot shows a VS Code editor with a Python script named `publisher_wildcard_plus.py`. The script uses the `paho.mqtt` library to connect to a local MQTT broker on `localhost:1883`. It publishes three messages: "Room 1 Temperature: 28 C", "Room 2 Temperature: 30 C", and "Room 1 Humidity: 65 %". A text editor window in the foreground shows the name "NABILLA SUCI AMANDA" and the ID "235150301111044". The terminal at the bottom shows the command to run the script and its output, which matches the printed messages in the code.

```
code > publisher_wildcard_plus.py > ...
1 import paho.mqtt.client as mqtt
2 import time
3
4 client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
5
6 client.connect("localhost", 1883, 60)
7 client.loop_start()
8
9 client.publish("smartroom/room1/temperature", "Room 1 Temperature: 28 C")
10 print("Room 1 temperature sent")
11 time.sleep(1)
12
13 client.publish("smartroom/room2/temperature", "Room 2 Temperature: 30 C")
14 print("Room 2 temperature sent")
15 time.sleep(1)
16
17 client.publish("smartroom/room1/humidity", "Room 1 Humidity: 65 %")
18 print("Room 1 humidity sent")
19
20 time.sleep(1)
21
22 client.loop_stop()
23 client.disconnect()
```

PS C:\Users\Nabilla Suci Amanda\Downloads\mqtt-smartroom-nabilla> & "C:\Users\Nabilla Suci Amanda\AppData\Local\Programs\Python\Python314\python.exe" "C:\Users\Nabilla Suci Amanda\Downloads\mqtt-smartroom-nabilla\code\publisher_wildcard_plus.py"

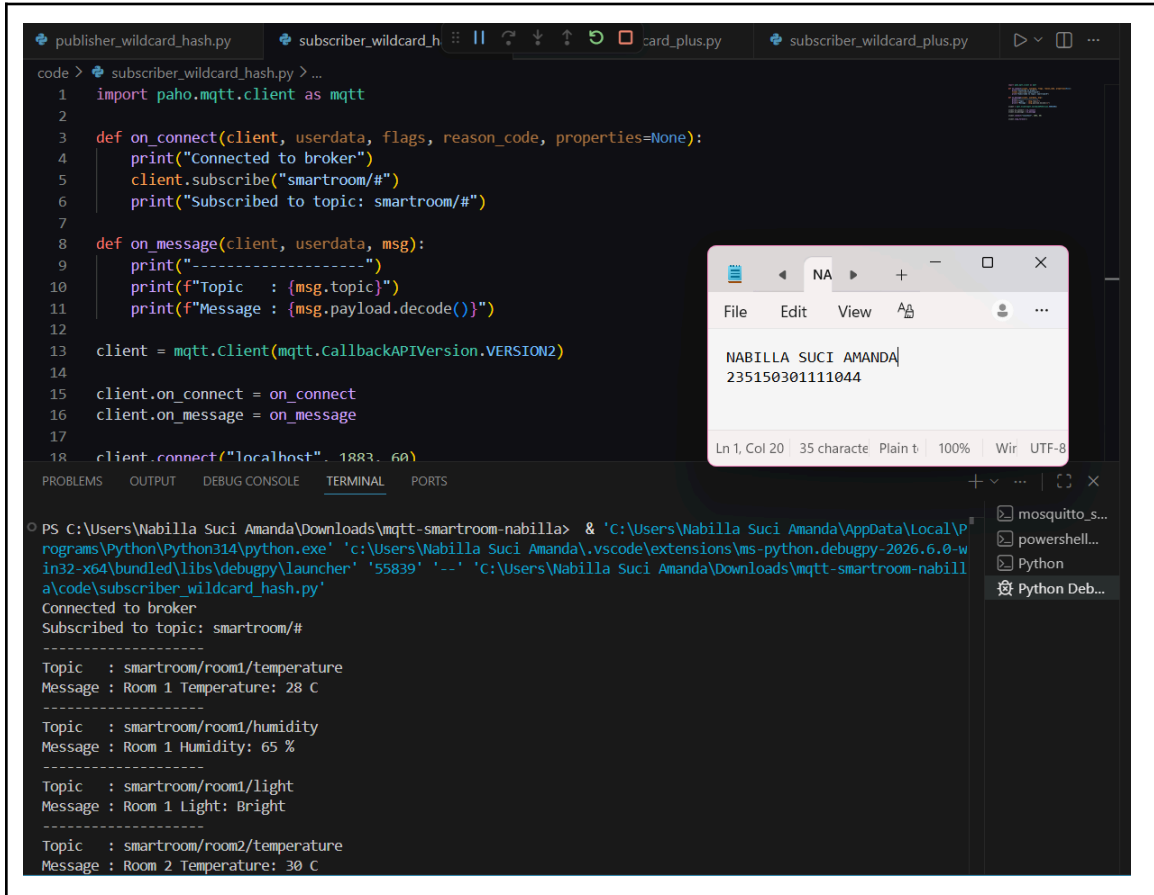
Room 1 temperature sent
Room 2 temperature sent
Room 1 humidity sent

PS C:\Users\Nabilla Suci Amanda\Downloads\mqtt-smartroom-nabilla>

5. Pengujian Skenario 5 : Wildcard Topic (#)

Pada pengujian terakhir digunakan wildcard # dengan topik subscribe smartroom/#. Publisher mengirimkan beberapa jenis data berupa temperature, humidity, dan light dari beberapa topik. Hasil pengujian menunjukkan bahwa subscriber berhasil menerima seluruh pesan yang berada di bawah domain smartroom, termasuk data dari room1 dan room2. Hal ini menunjukkan bahwa wildcard # mampu digunakan untuk monitoring seluruh sistem tanpa perlu melakukan subscribe pada masing-masing topik secara individual.

a. Hasil Pengujian Subscriber pada Skenario 5 : Wildcard Topic (#)



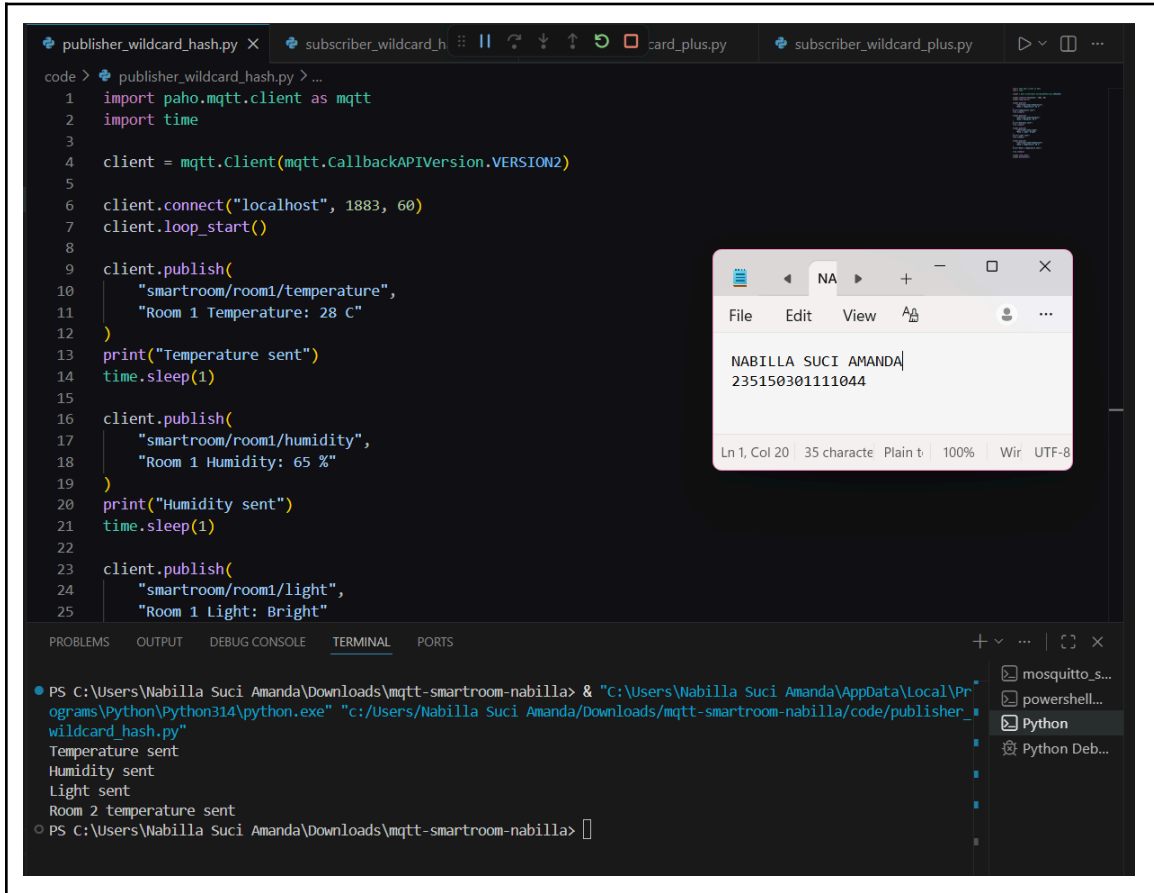
The screenshot displays a Visual Studio Code editor window with a Python file named `subscriber_wildcard_hash.py`. The code defines an MQTT client that connects to a broker at `localhost:1883` and subscribes to the `smartroom/#` topic. It prints the topic and message payload for each received message. The terminal output shows the client successfully connecting and receiving four messages from the `smartroom/room1` and `smartroom/room2` topics.

```
1 import paho.mqtt.client as mqtt
2
3 def on_connect(client, userdata, flags, reason_code, properties=None):
4     print("Connected to broker")
5     client.subscribe("smartroom/#")
6     print("Subscribed to topic: smartroom/#")
7
8 def on_message(client, userdata, msg):
9     print("-----")
10    print(f"Topic : {msg.topic}")
11    print(f"Message : {msg.payload.decode()}")
12
13 client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
14
15 client.on_connect = on_connect
16 client.on_message = on_message
17
18 client.connect("localhost", 1883, 60)
```

Terminal Output:

```
PS C:\Users\Nabilla Suci Amanda\Downloads\mqtt-smartroom-nabilla> & 'C:\Users\Nabilla Suci Amanda\AppData\Local\Programs\Python\Python314\python.exe' 'C:\Users\Nabilla Suci Amanda\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '55839' '--' 'C:\Users\Nabilla Suci Amanda\Downloads\mqtt-smartroom-nabilla\code\subscriber_wildcard_hash.py'
Connected to broker
Subscribed to topic: smartroom/#
-----
Topic : smartroom/room1/temperature
Message : Room 1 Temperature: 28 C
-----
Topic : smartroom/room1/humidity
Message : Room 1 Humidity: 65 %
-----
Topic : smartroom/room1/light
Message : Room 1 Light: Bright
-----
Topic : smartroom/room2/temperature
Message : Room 2 Temperature: 30 C
```

b. Hasil Pengujian Publisher pada Skenario 5 : Wildcard Topic (#)



The image shows a Visual Studio Code editor window with a Python script named `publisher_wildcard_hash.py`. The script uses the `paho.mqtt.client` library to connect to a local MQTT broker on `localhost:1883`. It publishes three messages to the topic `smartroom/room1/temperature`: "Room 1 Temperature: 28 C", "Room 1 Humidity: 65 %", and "Room 1 Light: Bright". A text editor window is overlaid on the script, showing the name `NABILLA SUCI AMANDA` and the ID `235150301111044`. Below the editor, a terminal window shows the command to run the script and its output, which matches the messages published by the script.

```
code > publisher_wildcard_hash.py > ...
1 import paho.mqtt.client as mqtt
2 import time
3
4 client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
5
6 client.connect("localhost", 1883, 60)
7 client.loop_start()
8
9 client.publish(
10     "smartroom/room1/temperature",
11     "Room 1 Temperature: 28 C"
12 )
13 print("Temperature sent")
14 time.sleep(1)
15
16 client.publish(
17     "smartroom/room1/humidity",
18     "Room 1 Humidity: 65 %"
19 )
20 print("Humidity sent")
21 time.sleep(1)
22
23 client.publish(
24     "smartroom/room1/light",
25     "Room 1 Light: Bright"
26 )
```

Terminal Output:

```
PS C:\Users\Nabilla Suci Amanda\Downloads\mqtt-smartroom-nabilla> & "C:\Users\Nabilla Suci Amanda\AppData\Local\Programs\Python\Python314\python.exe" "C:/Users/Nabilla Suci Amanda/Downloads/mqtt-smartroom-nabilla/code/publisher_wildcard_hash.py"
Temperature sent
Humidity sent
Light sent
Room 2 temperature sent
PS C:\Users\Nabilla Suci Amanda\Downloads\mqtt-smartroom-nabilla>
```

F. ANALISIS

Berdasarkan hasil pengujian yang telah dilakukan, komunikasi MQTT menggunakan Python dan Mosquitto Broker berhasil diimplementasikan pada studi kasus Smart Room Monitoring. Seluruh skenario pengujian menunjukkan bahwa pola komunikasi publish subscribe mampu berjalan dengan baik melalui perantara broker tanpa memerlukan hubungan langsung antara publisher dan subscriber. Mekanisme ini menunjukkan karakteristik MQTT yang fleksibel dan sesuai digunakan pada sistem monitoring berbasis Cyber Physical System. Pada skenario komunikasi dasar publisher subscriber, publisher berhasil mengirimkan data temperature menuju broker dan subscriber berhasil menerima pesan yang dikirimkan. Hal ini membuktikan bahwa Mosquitto Broker mampu menjalankan fungsi utamanya sebagai message broker yang menerima dan mendistribusikan pesan berdasarkan topik MQTT yang digunakan.

Pengujian menggunakan Quality of Service (QoS) menunjukkan adanya perbedaan perilaku pengiriman pesan. QoS 0 bekerja dengan mekanisme at most once, yaitu pesan dikirim tanpa proses konfirmasi sehingga memiliki overhead komunikasi paling kecil tetapi tidak menjamin pesan diterima. QoS 1 menggunakan mekanisme at least once sehingga pesan dijamin sampai meskipun memungkinkan terjadinya pengiriman ulang atau duplikasi. Sementara itu, QoS 2 menerapkan mekanisme exactly once melalui proses handshake yang lebih kompleks sehingga pesan diterima tepat satu kali. Berdasarkan pengujian, seluruh level QoS berhasil diterima oleh subscriber dengan benar, meskipun QoS yang lebih tinggi memerlukan proses komunikasi yang lebih banyak.

Pada pengujian beberapa topik, MQTT menunjukkan kemampuan untuk menangani beberapa jenis data dalam satu sistem monitoring. Subscriber berhasil menerima data temperature, humidity, dan light yang dikirim melalui topik berbeda. Hal ini menunjukkan bahwa struktur topik MQTT memungkinkan pengelompokan data secara sistematis sehingga komunikasi menjadi lebih modular dan mudah dikelola. Pengujian wildcard topic juga menunjukkan karakteristik penting MQTT. Wildcard + hanya menggantikan satu level topik sehingga subscriber hanya menerima data temperature dari beberapa ruangan dan tidak menerima data humidity yang tidak sesuai pola subscribe. Sebaliknya, wildcard # mampu menerima seluruh subtopik di bawah domain smartroom, sehingga seluruh data monitoring berhasil diterima tanpa perlu subscribe pada setiap topik secara individual. Mekanisme ini sangat bermanfaat pada sistem monitoring berskala besar karena dapat menyederhanakan proses pengawasan data. Secara keseluruhan, hasil pengujian menunjukkan bahwa MQTT memiliki keunggulan berupa komunikasi yang ringan, fleksibel, dan mudah dikembangkan untuk kebutuhan Cyber Physical System maupun Internet of Things. Penggunaan broker, struktur topik, QoS, serta wildcard topic memberikan fleksibilitas tinggi dalam pengelolaan komunikasi data pada sistem monitoring seperti Smart Room Monitoring.

G. KESIMPULAN

Berdasarkan implementasi dan pengujian yang telah dilakukan, komunikasi MQTT menggunakan Python dan Mosquitto Broker pada sistem Smart Room Monitoring berhasil dijalankan dengan baik. Sistem mampu menerapkan pola komunikasi publish subscribe, di mana publisher mengirimkan data sensor virtual menuju broker dan subscriber menerima data sesuai topik yang di-subscribe. Pengujian pada lima skenario menunjukkan bahwa komunikasi dasar publisher–subscriber dapat berjalan dengan baik melalui Mosquitto Broker. Penggunaan Quality of Service (QoS) berhasil menunjukkan perbedaan tingkat jaminan pengiriman pesan, mulai dari QoS 0 yang ringan tanpa konfirmasi, QoS 1 yang menjamin pesan sampai minimal satu kali, hingga QoS 2 yang memastikan pesan diterima tepat satu kali.

Selain itu, penggunaan beberapa topik menunjukkan bahwa MQTT mampu menangani berbagai jenis data monitoring secara terstruktur. Pengujian wildcard + dan # juga membuktikan bahwa MQTT menyediakan mekanisme subscribe yang fleksibel untuk monitoring data berdasarkan pola topik tertentu maupun seluruh domain topik. Secara keseluruhan, MQTT terbukti menjadi protokol komunikasi yang ringan, fleksibel, dan efektif untuk diterapkan pada Cyber Physical System dan Internet of Things, khususnya pada sistem monitoring seperti Smart Room Monitoring.

Link Github :

https://github.com/nabsuc/MQTT_smartroom_CPS_Nabilla-Suci-Amanda.git